Coding Agent

Do / Say / See

Plan a focused SQL fix, approve the approach, steer the work, inspect the patch and publish a PR.

5 pillars | 20 core pages | limit 25

Use demo-site-pr-recordings / codex/bulk-checkout-flagship. Follow recorded task 19bb8d53; its delivered result is PR #10. The original finding remains in demo-site-pr-examples PR #4.

Read each grouped step top to bottom. Do contains the clicks and scrolls; read only Say aloud. See is a frame from the delivered video. Times refer to that video. The narration is preserved in full, including the recorded opening and closing.

Creation, Send and approval actions describe the recorded flow. Rehearse with the linked existing results; use a separate training run when repeating those actions. Live pages may have changed. Resource links require the corresponding account access.

OPEN THE DEMO

PR #4: SQL finding

Recording repository

Recorded Coding Agent task

Coding Agent result: PR #10

PILLARS AND PAGE LIMITS

Introduction - 1 page

Start a Task - 3 steps / pages

Pick the Mode - 4 steps / pages

Watch It Work - 4 steps / pages

Review the Patch - 4 steps / pages

Deliver - 4 steps / pages

The core count includes the opening page when present. This cover, operator prompts and resource directory are outside the core. Every pillar uses at most four grouped steps.

INTRODUCTION / OPENING / 00:00-00:23

Introduce the flow

DO

Show the section’s flow slide. Keep the full slide visible and highlight each pillar as it comes up in the introduction.

At the end of the introduction, switch to the first demo screen.

Slide deck

SAY

I'll give the coding agent a focused task. Fix the invoice search vulnerability while preserving normal search behavior. We'll choose the repository and branch, select plan mode and review the approach before implementation. As the agent works, I'll follow its progress and add a test case. Then we'll inspect the patch and validation and open a pull request so the team can review the result.

SEE / VIDEO FRAME AT 00:09



START A TASK / STEP 1 OF 3 / 00:25-00:40

Choose the repository and branch

DO

Open New coding task, then choose coderabbit-demo/demo-site-pr-recordings and codex/bulk-checkout-flagship.

Keep the original SQL finding available for context. For rehearsal, use the completed task.

Recorded Coding Agent task

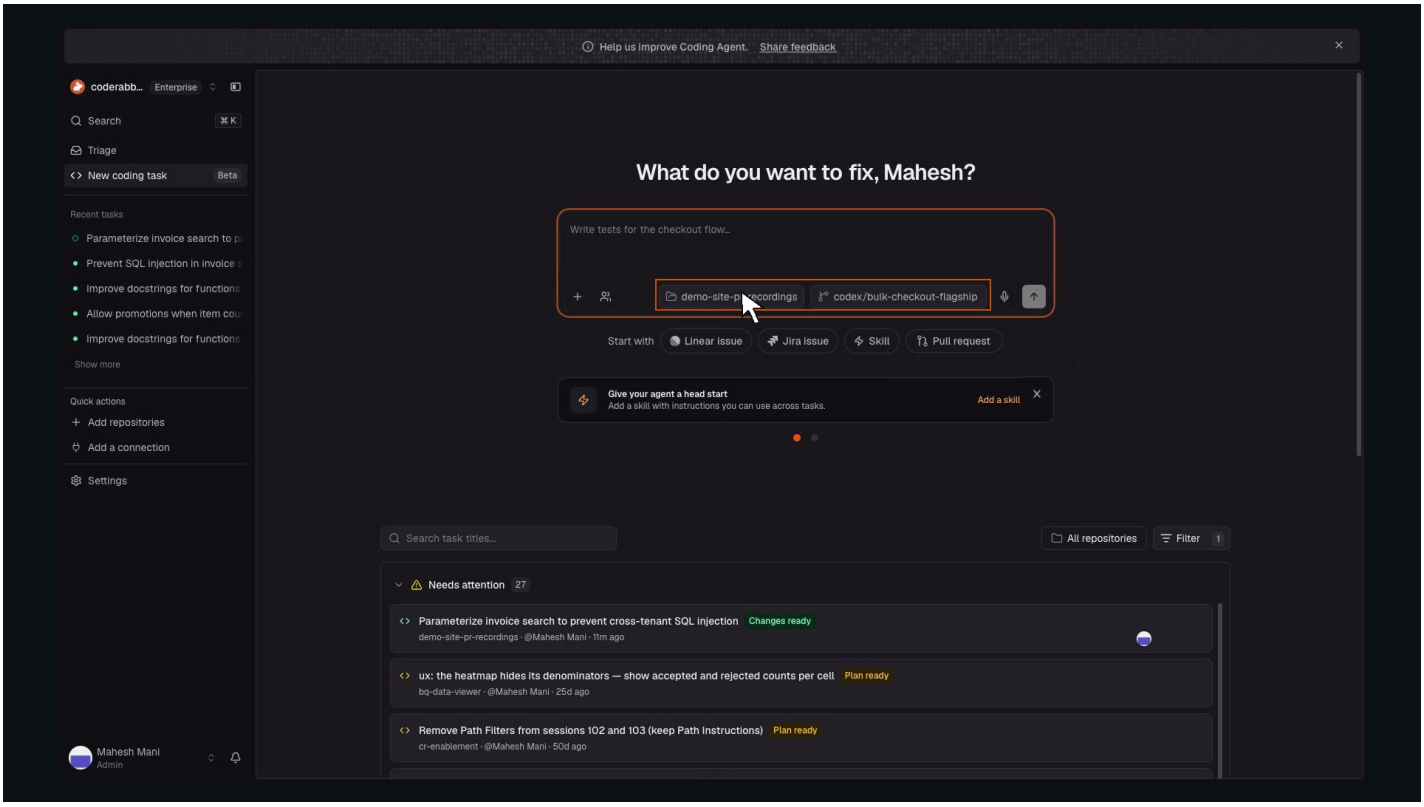
Recording repository

PR #4: SQL finding

SAY

I open a coding task and select the repository and branch containing our invoice search feature. The problem is that search text is inserted directly into a database command. A crafted search could change that command and retrieve another customer's invoices.

SEE / VIDEO FRAME AT 00:29



START A TASK / STEP 2 OF 3 / 00:40-01:07

Enter the focused request

DO

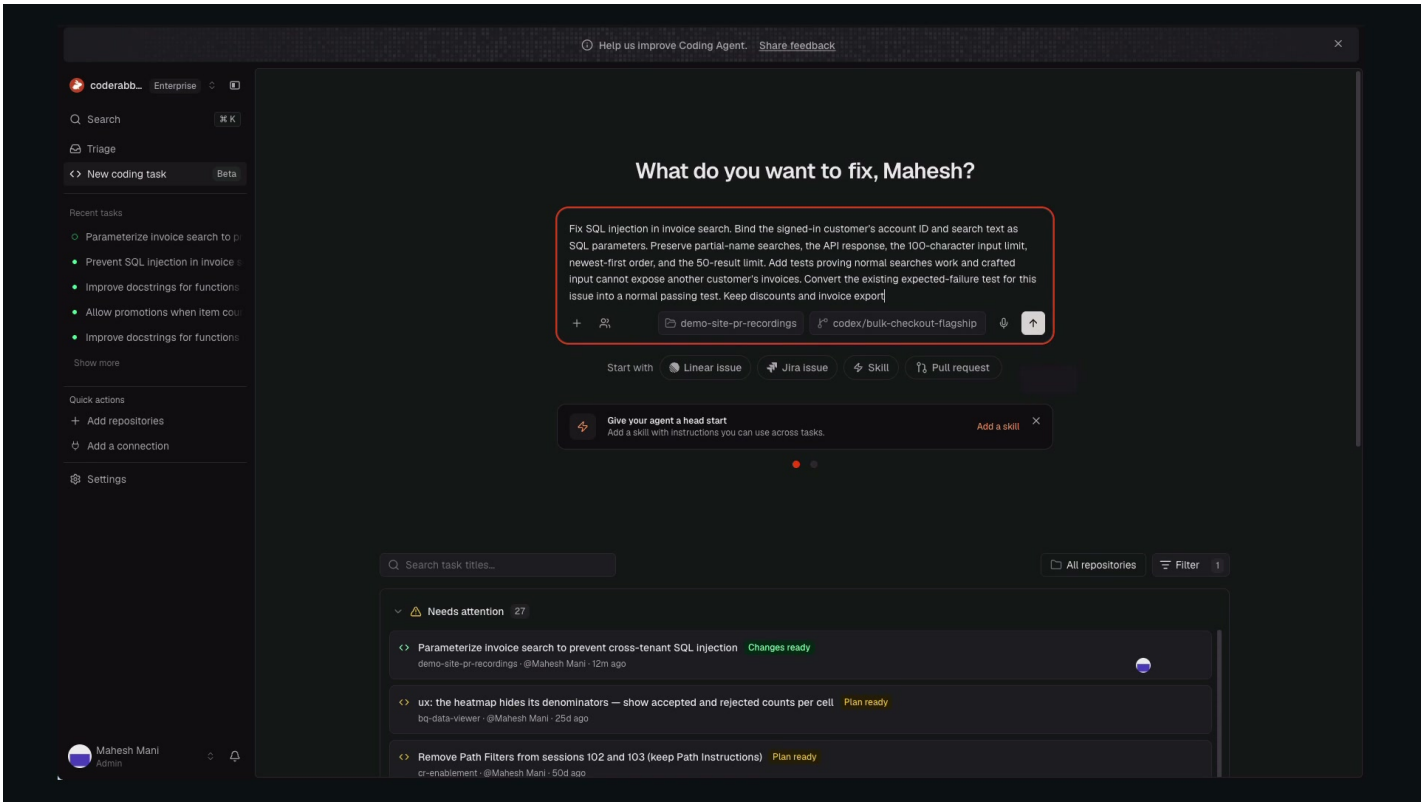
Click the task description and enter the Recorded coding-task request from the appendix.

Scroll within the field to verify tenant isolation, normal searches, order, limit, tests and excluded issues.

SAY

My request describes the result we need. Keep search results within the customer's account and treat search text as data. I also describe the behavior to preserve. Customers should still be able to search using part of a product name and results should keep their existing order and limit. I ask for regression tests that demonstrate the fix. This task covers invoice search and its tests with the discount and invoice export issues left for separate work.

SEE / VIDEO FRAME AT 01:00



START A TASK / STEP 3 OF 3 / 01:08-01:10

Keep the task ready for planning

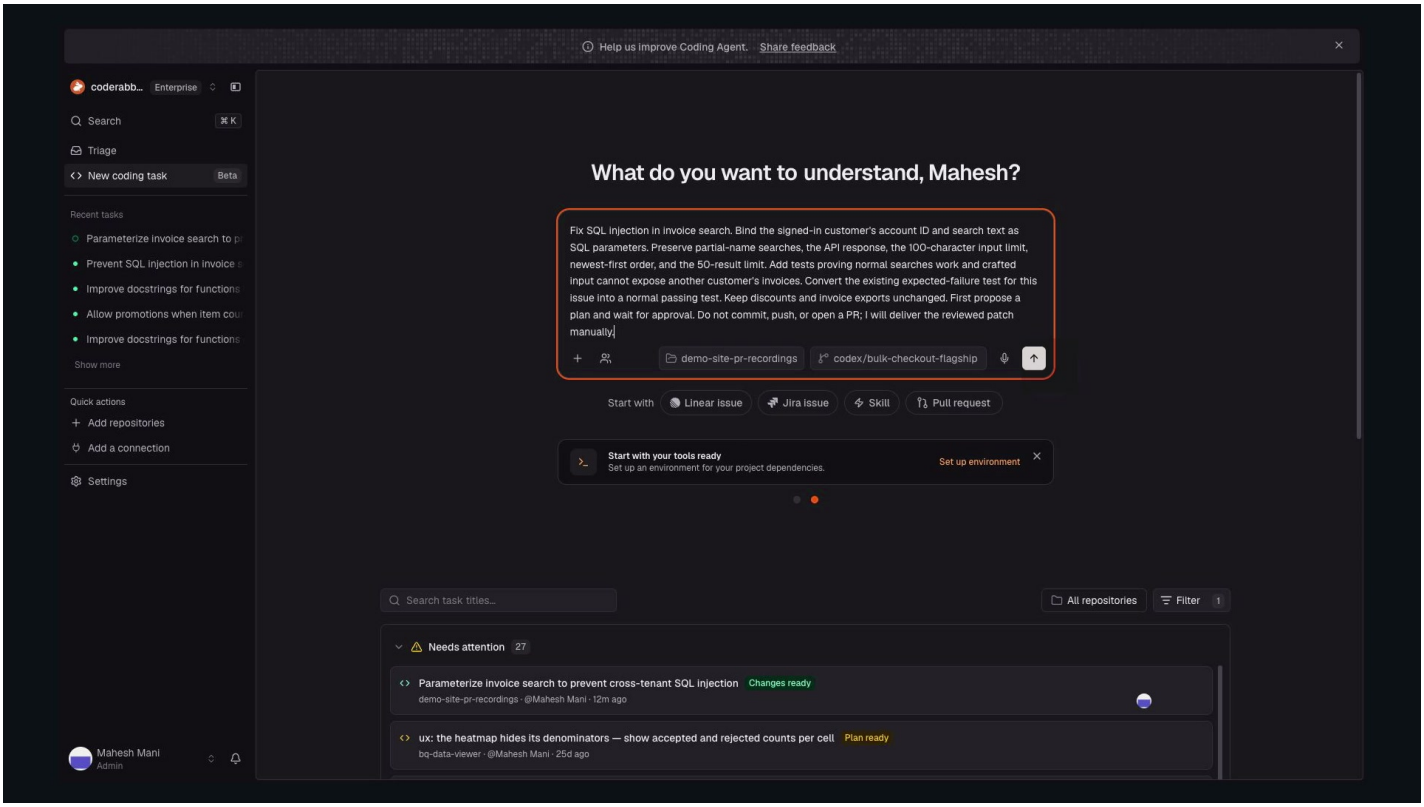
DO

Leave the composer unsubmitted while explaining that the approach will be reviewed first.

SAY

Before implementation, I want to review the approach.

SEE / VIDEO FRAME AT 01:10



PICK THE MODE / STEP 1 OF 4 / 01:10-01:21

Select Plan and send

DO

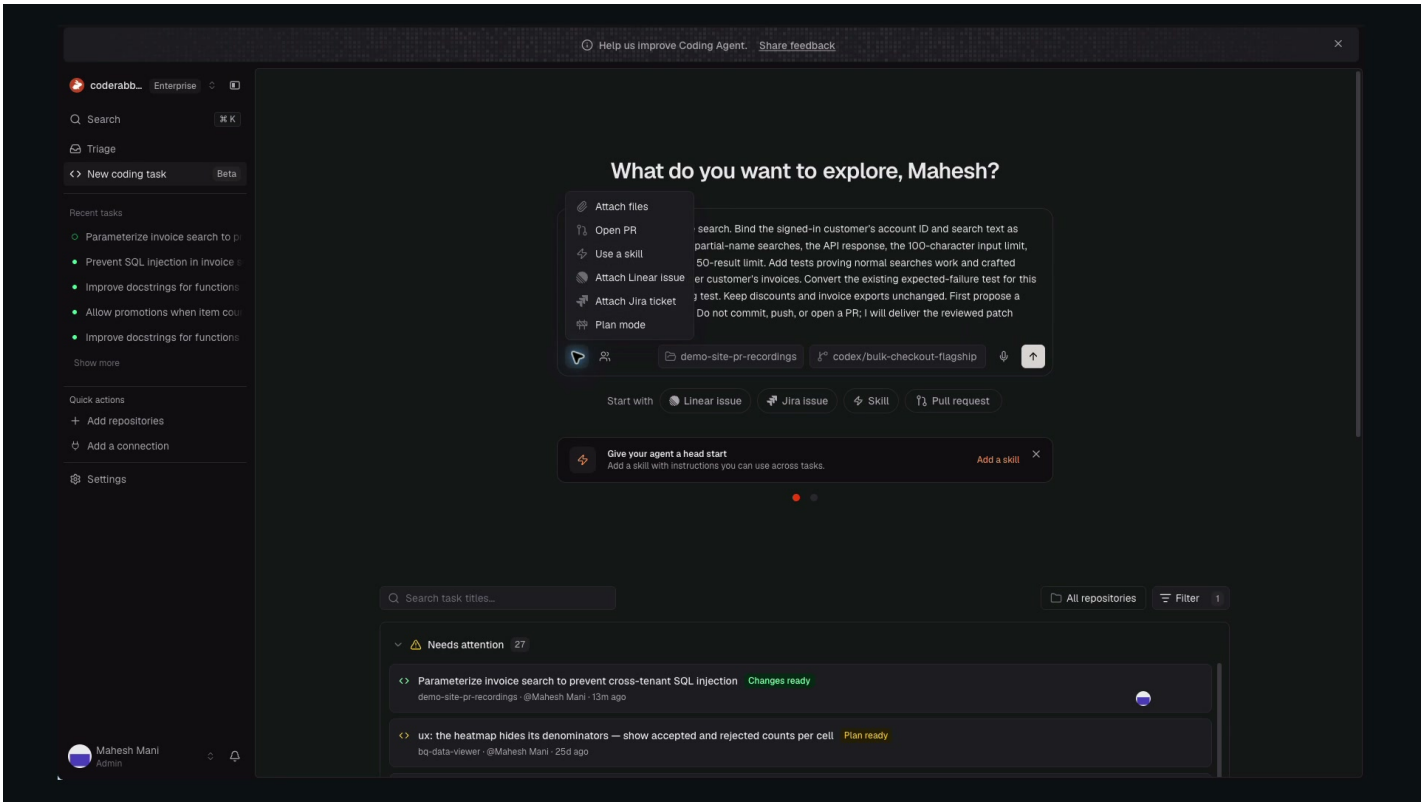
Click Add files and more beside the description, then Plan mode.

Confirm the Plan chip and click Start task. Wait for workspace setup.

SAY

I open the mode menu beside the task description and select plan. The label confirms that choice before I send the request. The agent reads the repository and proposes an approach.

SEE / VIDEO FRAME AT 01:14



PICK THE MODE / STEP 2 OF 4 / 01:21-01:42

Read the proposed plan

DO

Open the Plan card and scroll through Implementation, Tests, and Validation and Delivery.

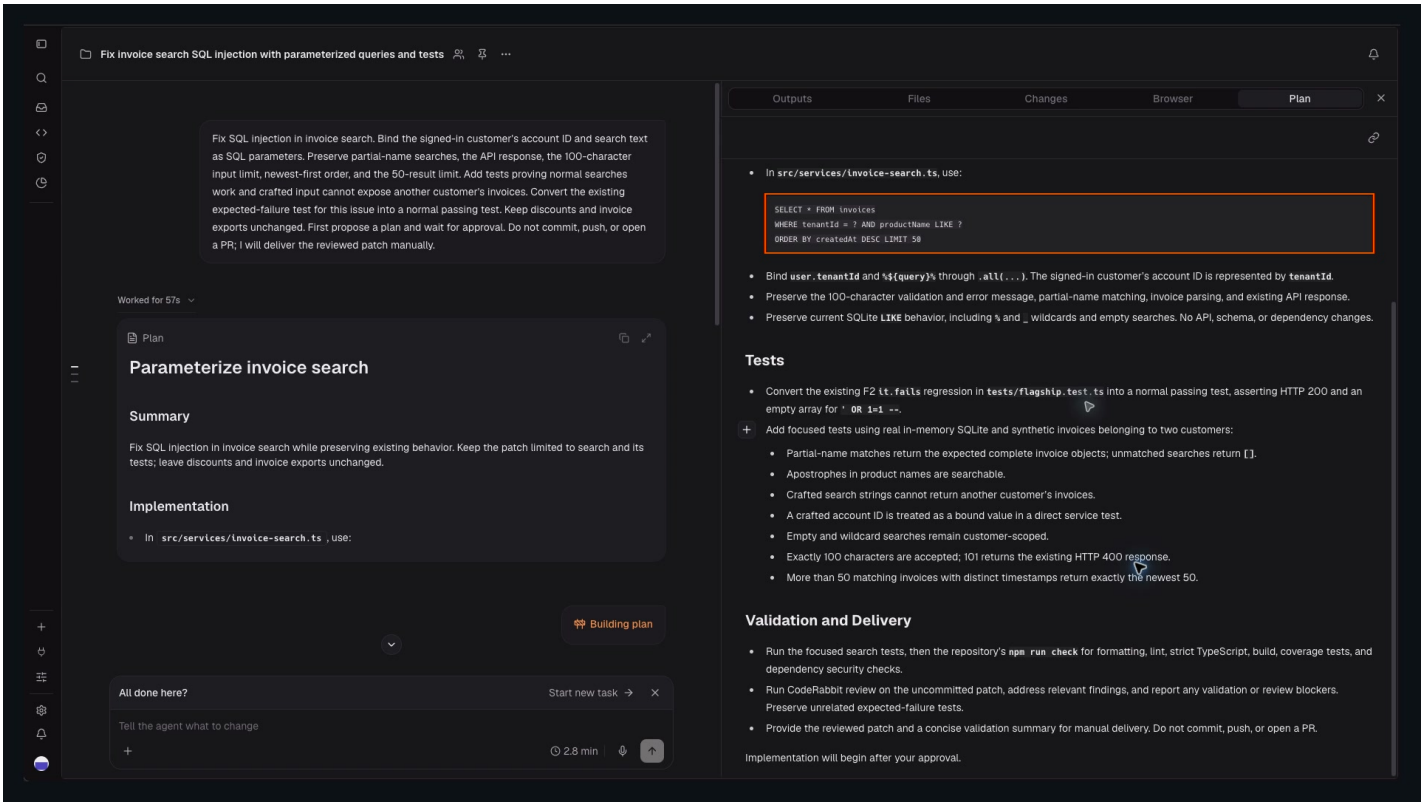
Check parameter binding, normal searches and tenant isolation against the original request.

Recorded Coding Agent task

SAY

I open the plan to check what it intends to change and how it will test the result. It proposes separating the search values from the SQL command, then checking both ordinary searches and attempts to cross the customer boundary. I compare that with my request. The plan needs to preserve useful search behavior as well as address the vulnerability.

SEE / VIDEO FRAME AT 01:33



PICK THE MODE / STEP 3 OF 4 / 01:42-01:48

Inspect the approval gate

DO

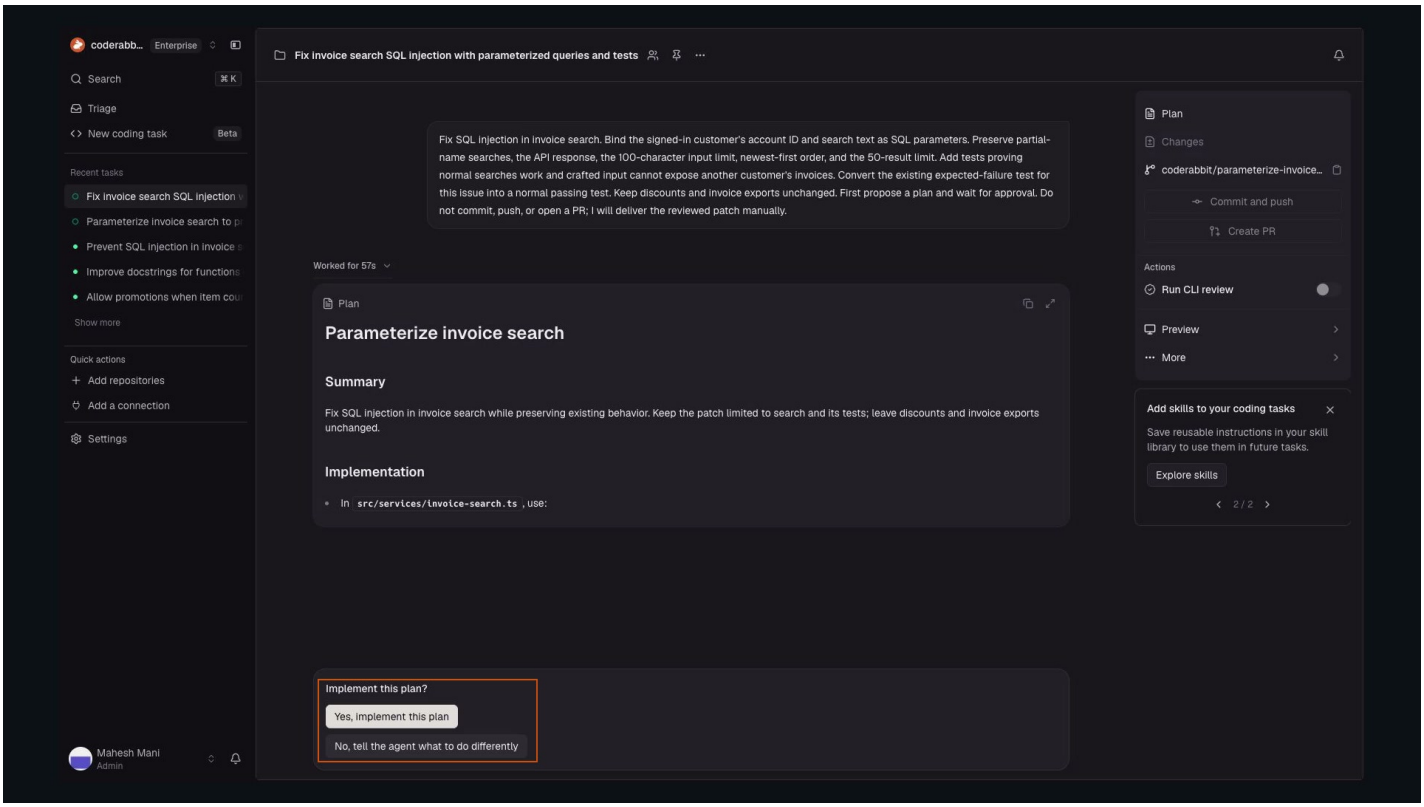
Scroll to Implement this plan? and show both implementation and revision choices.

Pause on the choice before continuing.

SAY

At the end, the agent asks whether to implement the plan. I can request changes here if the scope or approach needs work.

SEE / VIDEO FRAME AT 01:47



PICK THE MODE / STEP 4 OF 4 / 01:49-01:58

Approve the recorded approach

DO

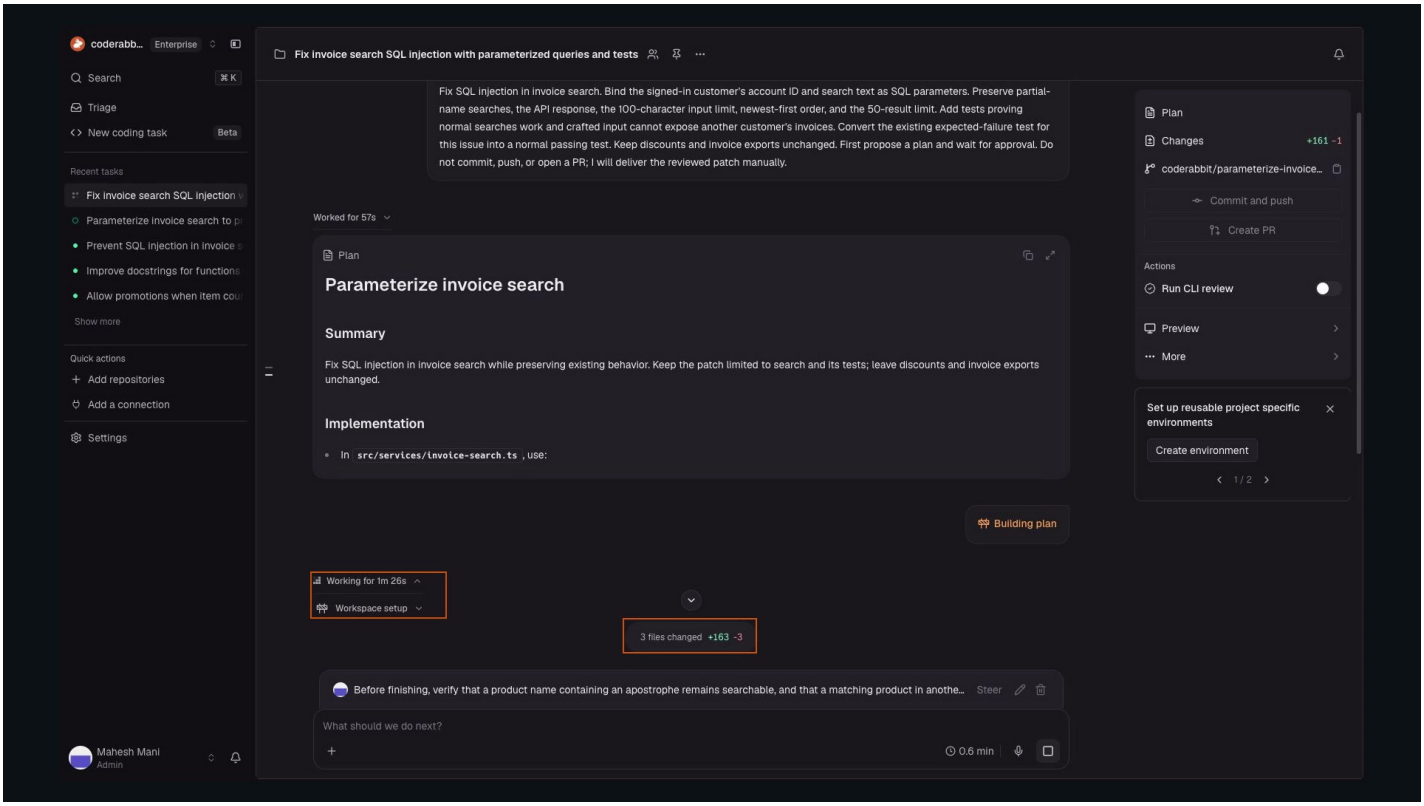
For a training run, click Yes, implement this plan once. For rehearsal, inspect the saved approval.

Show the activity entry and task branch as implementation begins.

SAY

For this task, I select yes, implement this plan. That starts the implementation in the task workspace. We'll inspect the finished patch before publishing it.

SEE / VIDEO FRAME AT 01:57



WATCH IT WORK / STEP 1 OF 4 / 01:59-02:12

Follow activity

DO

Open task activity and expand file/command entries.

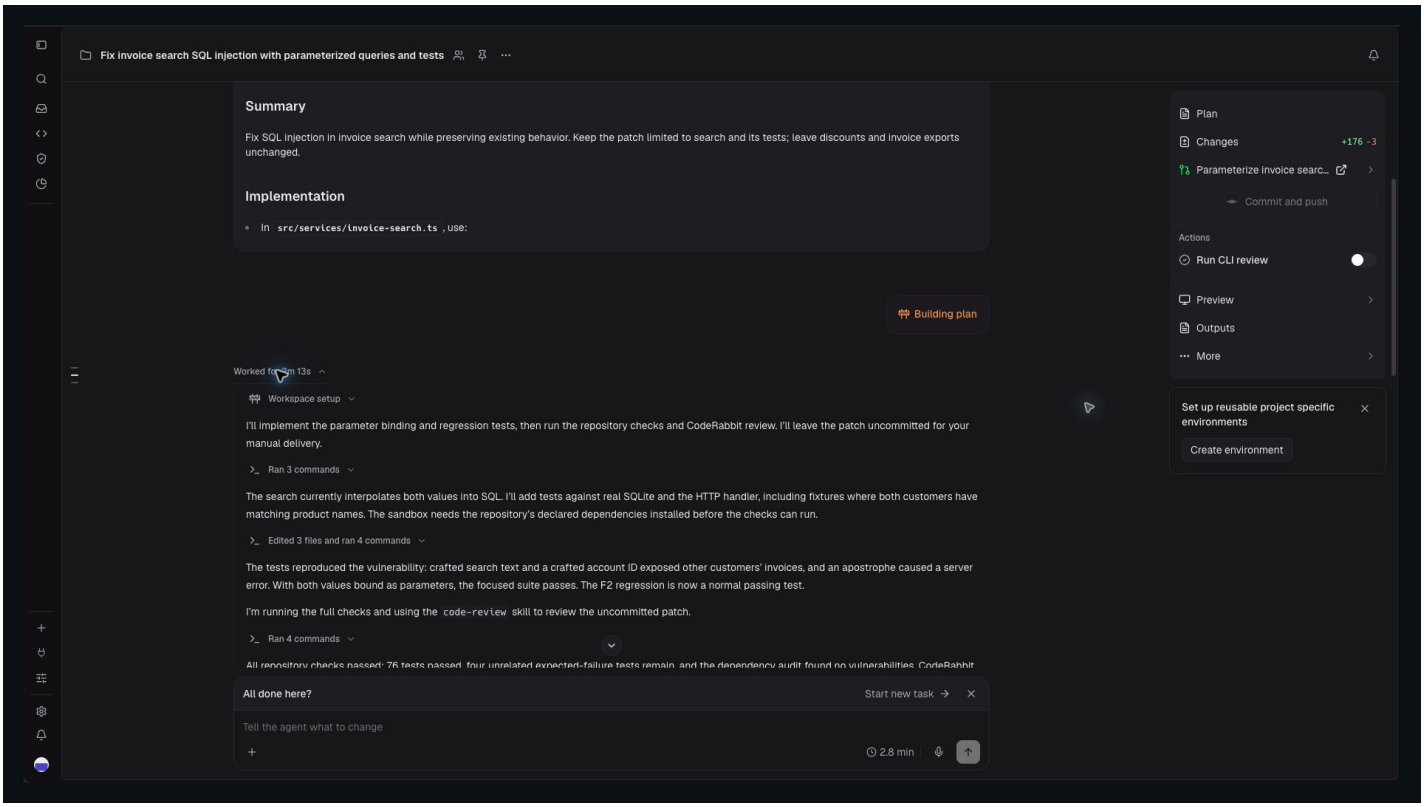
Scroll to invoice-search.ts and the regression-test work.

Recorded Coding Agent task

SAY

I open the task activity to follow the agent as it reads files, edits the code, and runs commands. The workspace keeps the implementation available for inspection while the agent works. These edits haven't yet been published to GitHub.

SEE / VIDEO FRAME AT 02:09



WATCH IT WORK / STEP 2 OF 4 / 02:13-02:29

Add the specific test request

DO

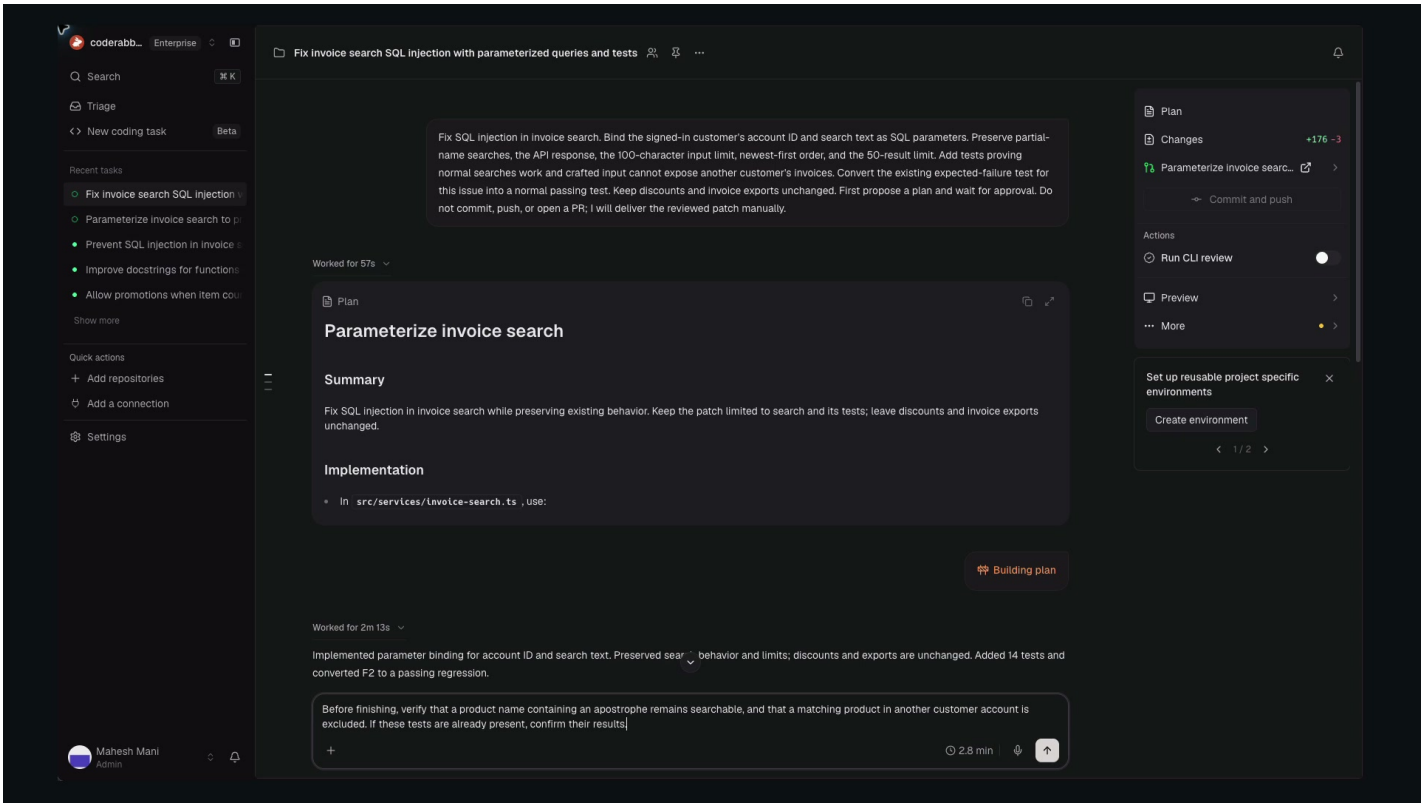
Click the follow-up input and enter the Mid-task test request from the appendix.

Click Send for a training run; for rehearsal, scroll to the recorded request.

SAY

I add a specific test request. Check a product name containing an apostrophe and check that another customer's matching invoice stays out of the result. Those are useful cases because the fix needs to preserve ordinary text searches while maintaining the customer boundary.

SEE / VIDEO FRAME AT 02:25



WATCH IT WORK / STEP 3 OF 4 / 02:30-02:35

Read the two-case response

DO

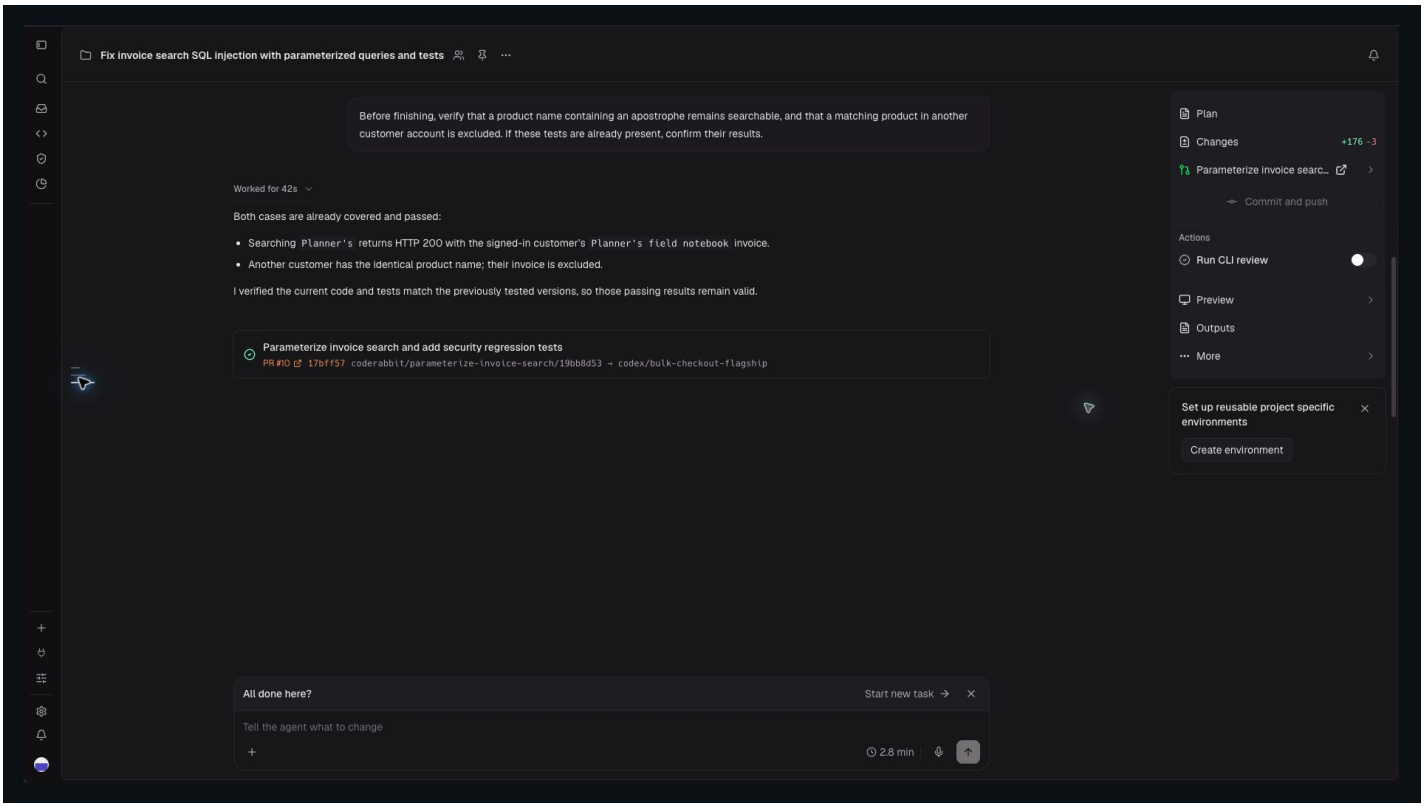
Scroll to the reply confirming the apostrophe search and other-customer exclusion.

Keep both results visible.

SAY

When the task finishes, I read the response. The agent confirms that both cases passed.

SEE / VIDEO FRAME AT 02:33



WATCH IT WORK / STEP 4 OF 4 / 02:35-02:47

Check validation and its limit

DO

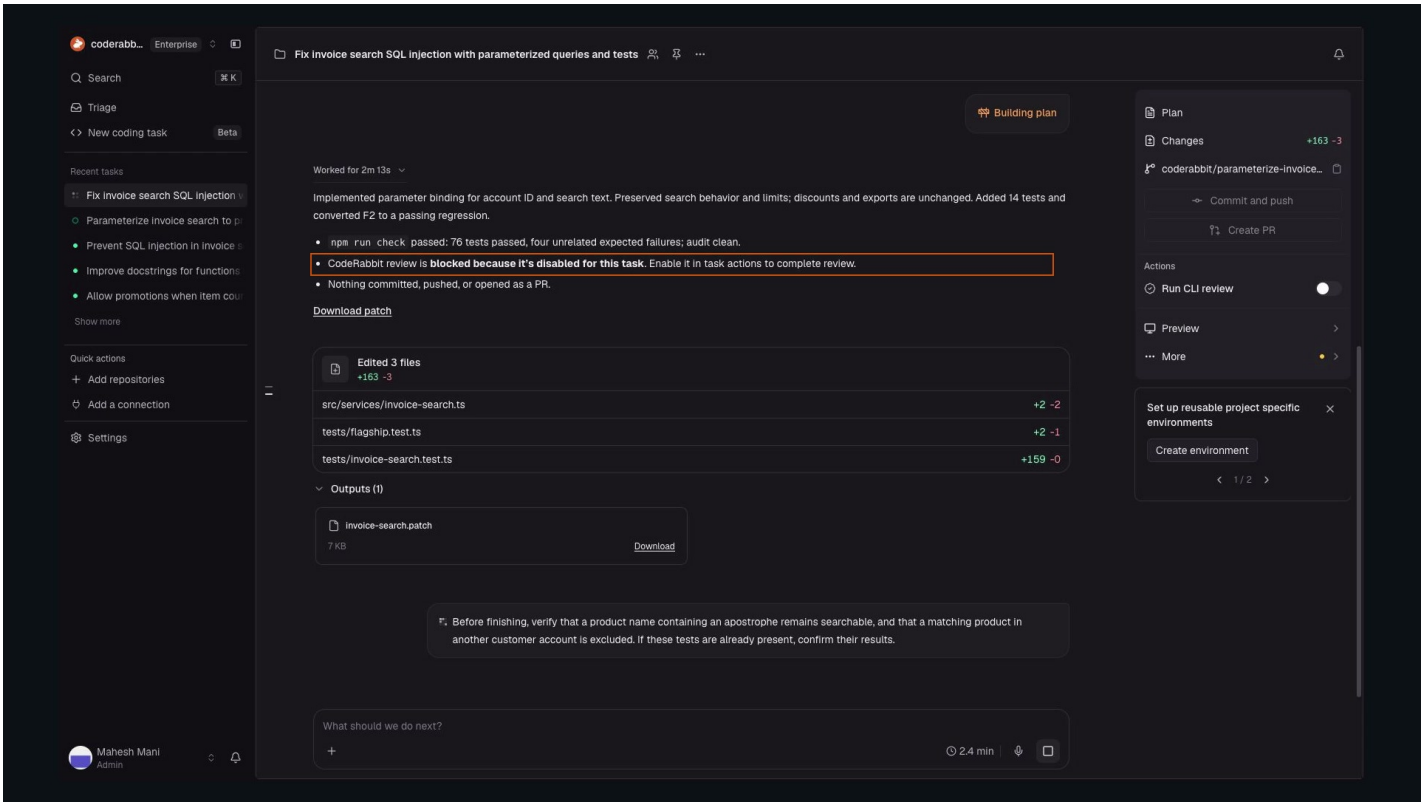
Scroll to the final task summary and dependency audit.

Read the note that optional CodeRabbit review was disabled before opening Changes.

SAY

It also reports the overall test results and dependency audit. The optional CodeRabbit review was disabled for this task, so it didn't run. I'll keep that in mind as I open the patch and inspect the validation myself.

SEE / VIDEO FRAME AT 02:44



REVIEW THE PATCH / STEP 1 OF 4 / 02:48-03:01

Inspect the SQL patch

DO

Click Changes or the Changed files summary. Select src/services/invoice-search.ts.

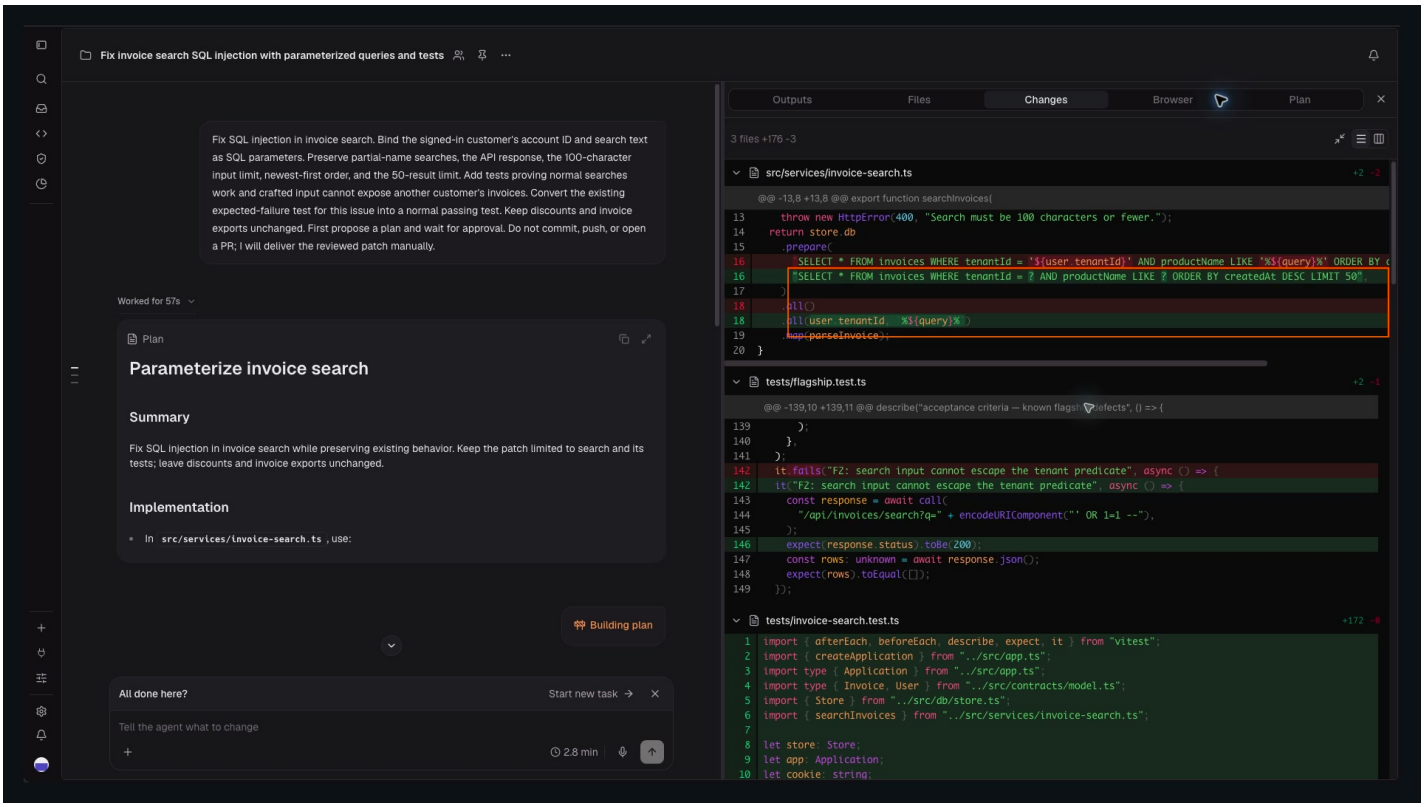
Scroll to the query placeholders and separately supplied tenant/search values.

Recorded Coding Agent task

SAY

I open the changed files and start with the invoice search query. The SQL command now uses placeholders. The customer account and search text are supplied separately, allowing the database to treat them as values.

SEE / VIDEO FRAME AT 02:59



REVIEW THE PATCH / STEP 2 OF 4 / 03:01-03:12

Inspect the security regression

DO

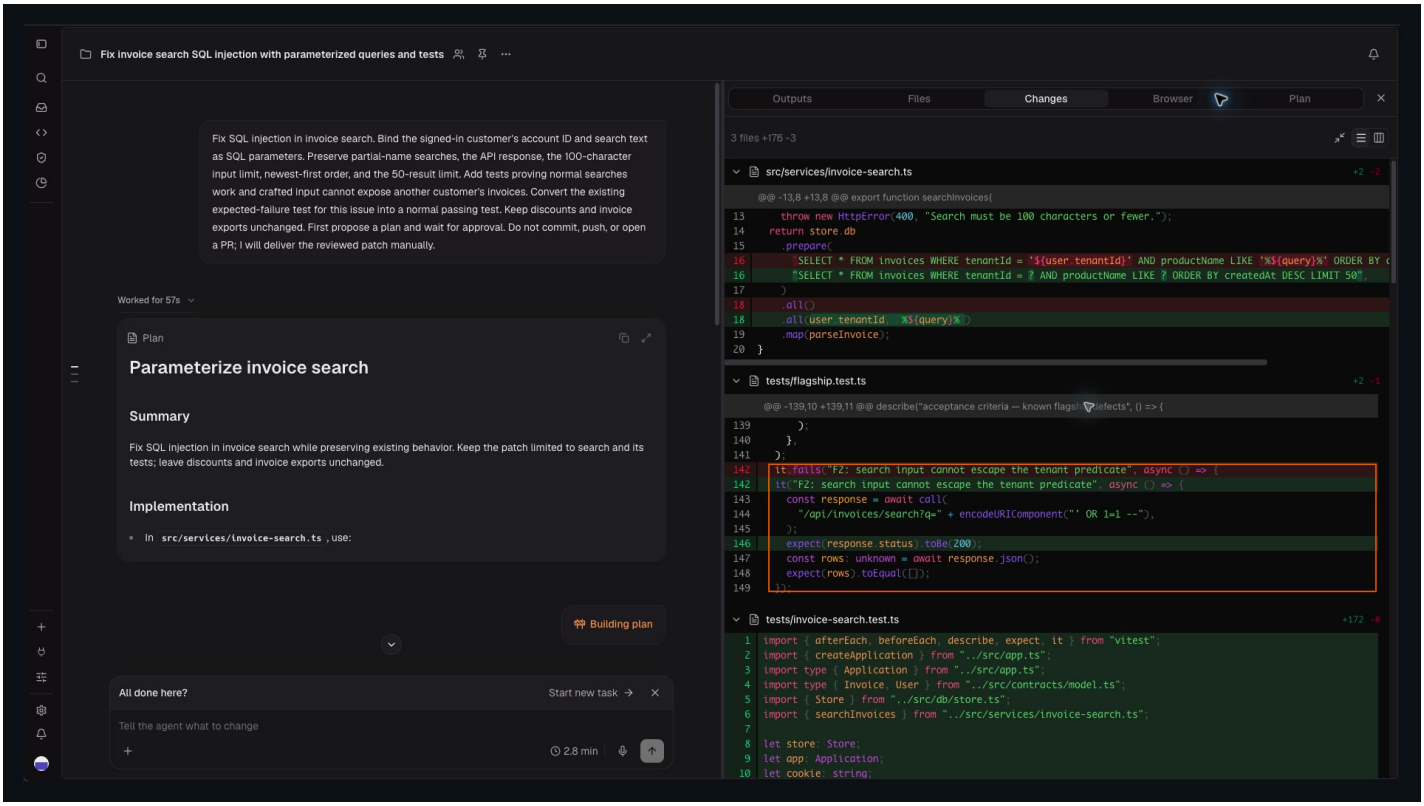
Select tests/flagship.test.ts in the file list.

Scroll to the former it.fails case and show the normal assertion for the crafted cross-tenant search.

SAY

Next, I open the regression tests. The test for the original security problem is now a normal assertion that must pass. It checks that a crafted search cannot retrieve another customer's invoices.

SEE / VIDEO FRAME AT 03:10



REVIEW THE PATCH / STEP 3 OF 4 / 03:13-03:21

Inspect normal search cases

DO

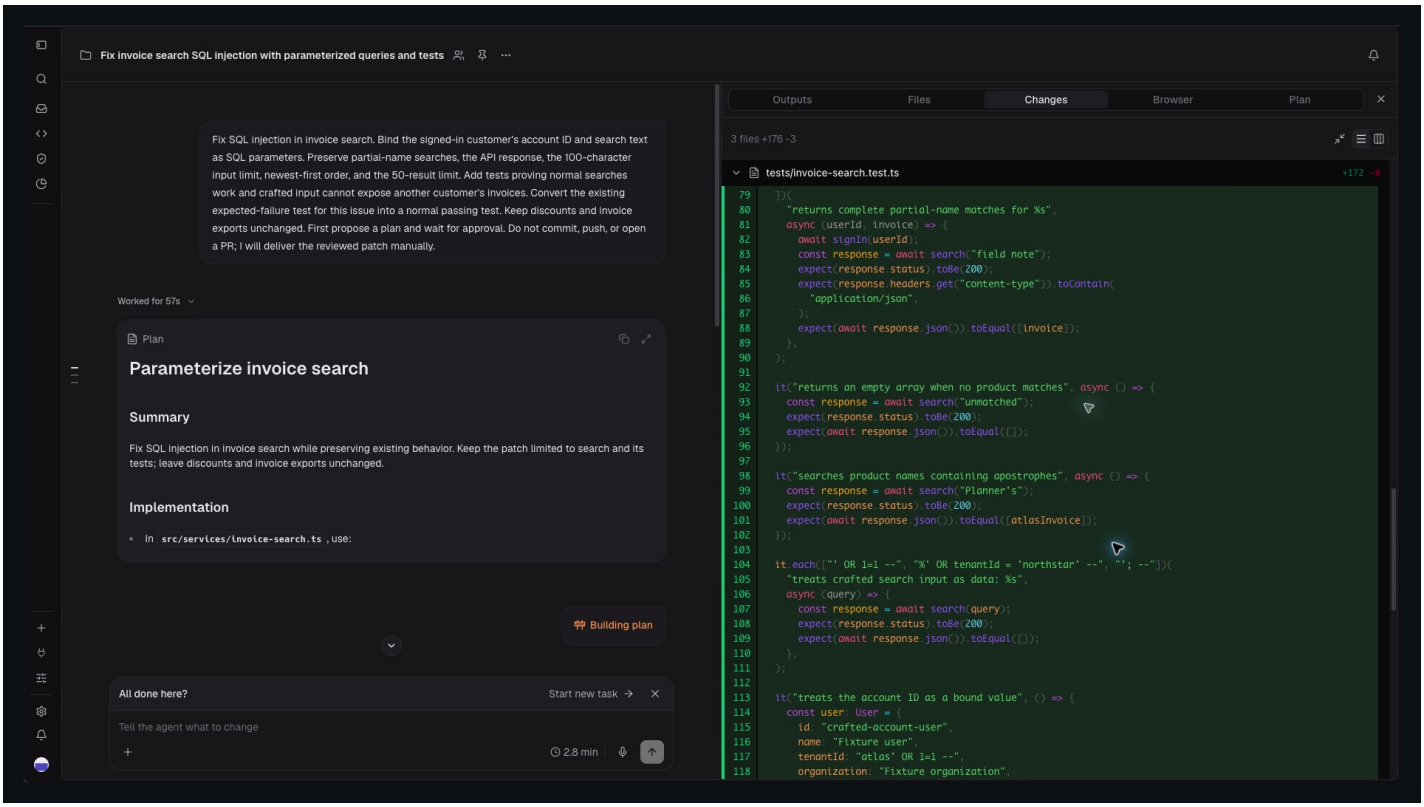
Select the invoice-search test file.

Scroll through partial names, apostrophe text, ordering and result-limit cases.

SAY

I also inspect the ordinary cases. Partial name searches, a product name with an apostrophe, the expected result order, and the result limit.

SEE / VIDEO FRAME AT 03:20



REVIEW THE PATCH / STEP 4 OF 4 / 03:23-03:44

Review the evidence before delivery

DO

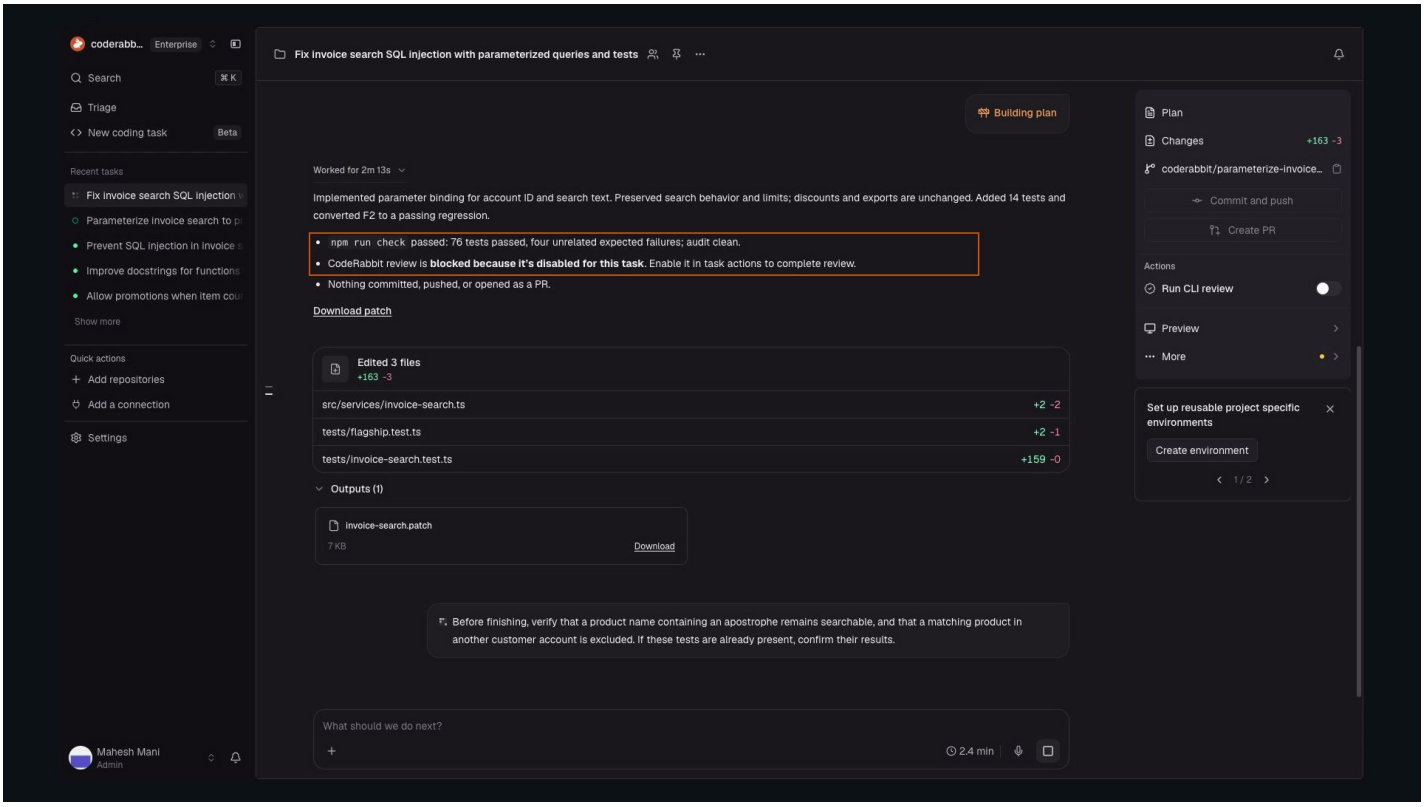
Return to the task summary. Read 76 passing tests, four expected failures and the dependency audit.

Keep the disabled-review note separate. Show the follow-up input where a revision could be requested.

SAY

The recorded task reports 76 passed tests and 4 unrelated expected failures along with a clean dependency audit. The additional CodeRabbit review didn't run, as we saw in the task settings. If the patch missed the requirement, I could ask the agent to revise it here. For this example, the code and tests address the requested behavior, so I'll move to delivery.

SEE / VIDEO FRAME AT 03:32



DELIVER / STEP 1 OF 4 / 03:45-03:58

Create the review PR

DO

Show Commit and push and Create PR. Choose Create PR for a training run and wait for publication.

For rehearsal, inspect the existing delivery result and open PR #10.

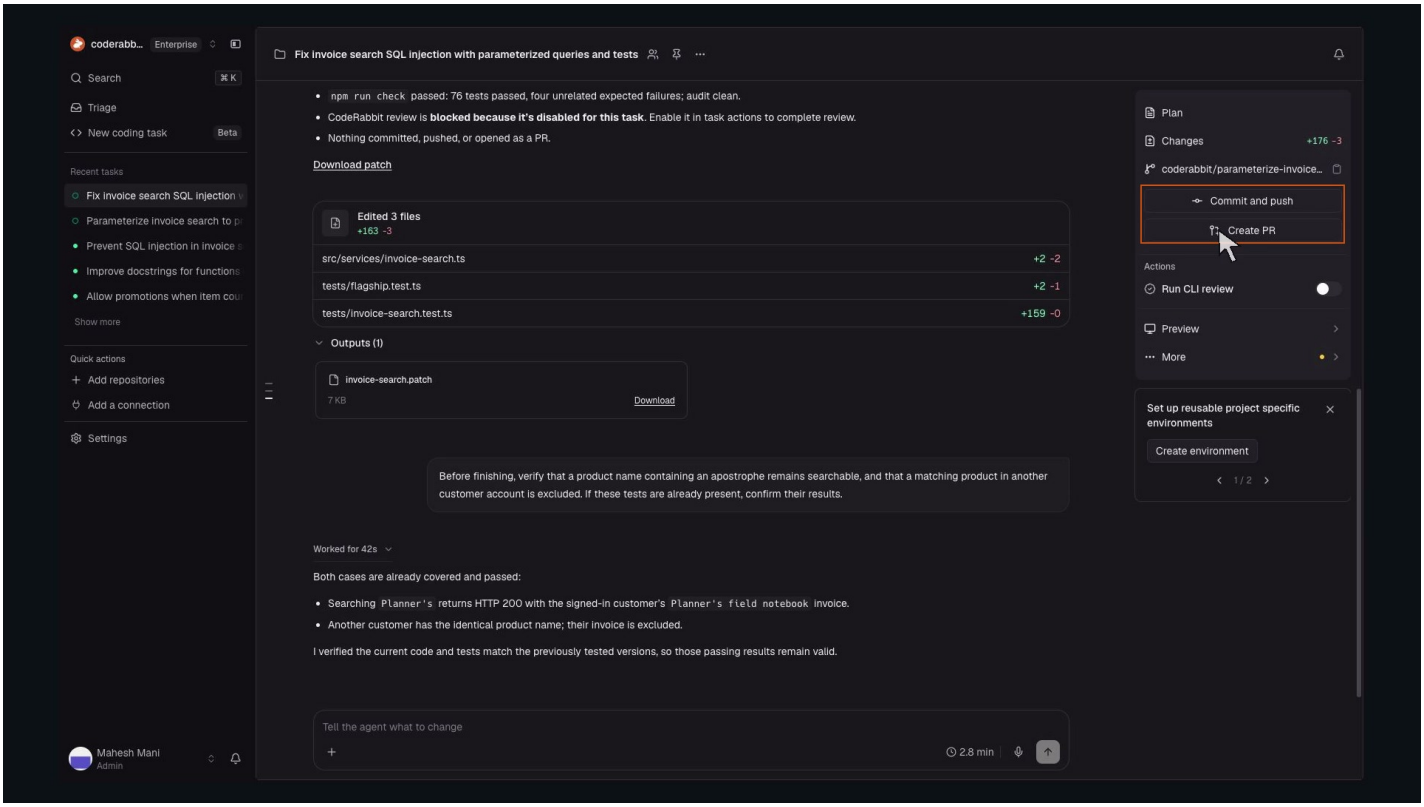
Recorded Coding Agent task

Coding Agent result: PR #10

SAY

The task offers commit and push or create PR. I choose create PR so the fix can have its own review. That action starts publishing the change. Once it finishes, I follow the link to GitHub and check the destination branch.

SEE / VIDEO FRAME AT 03:50



DELIVER / STEP 2 OF 4 / 03:59-04:05

Verify the destination

DO

Read PR #10's head/base line and confirm it targets the original feature branch.

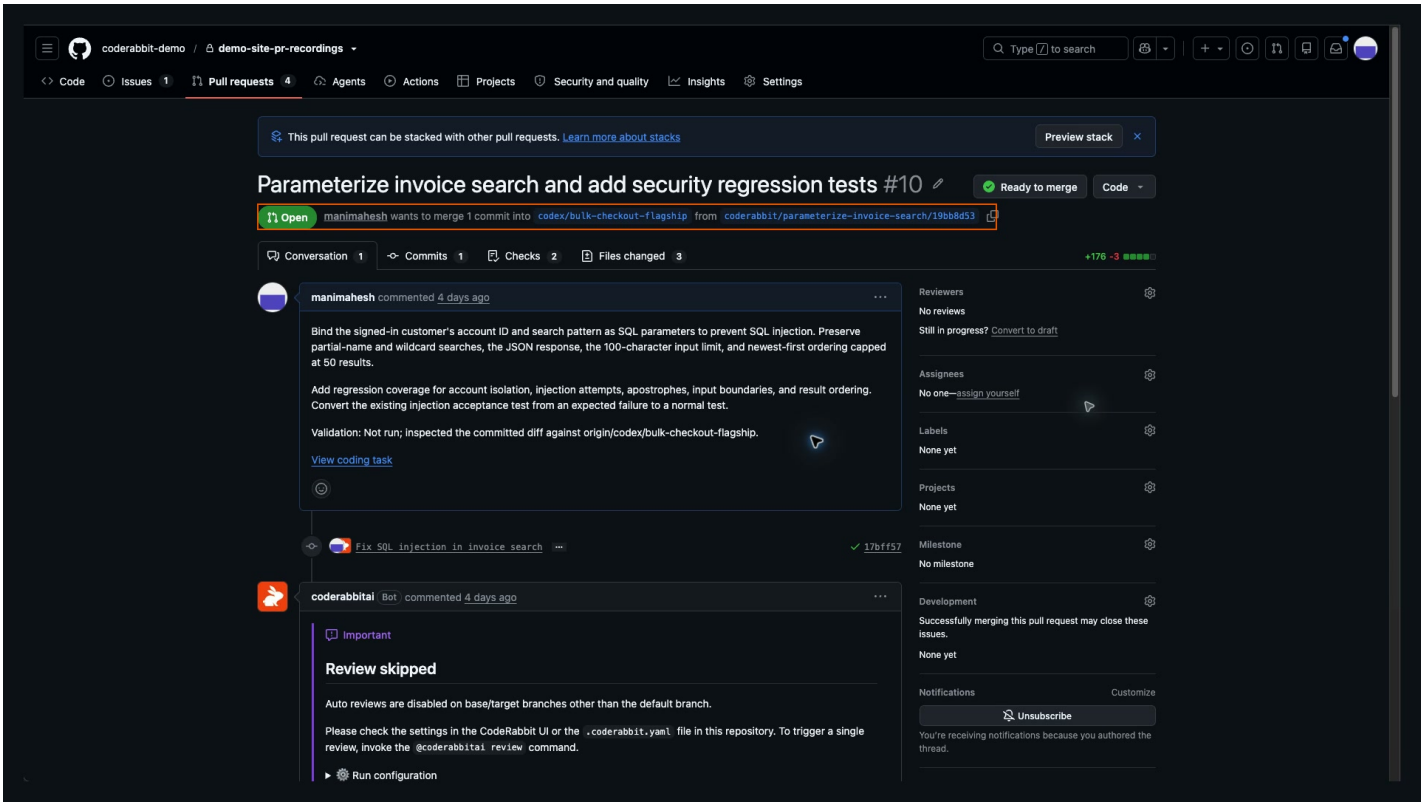
Keep the branch information visible.

Coding Agent result: PR #10

SAY

This fix targets our original feature branch, keeping the invoice search correction separate from the larger feature.

SEE / VIDEO FRAME AT 04:04



DELIVER / STEP 3 OF 4 / 04:06-04:15

Inspect the published files

DO

Click Files changed. Open the invoice-search query and regression tests.

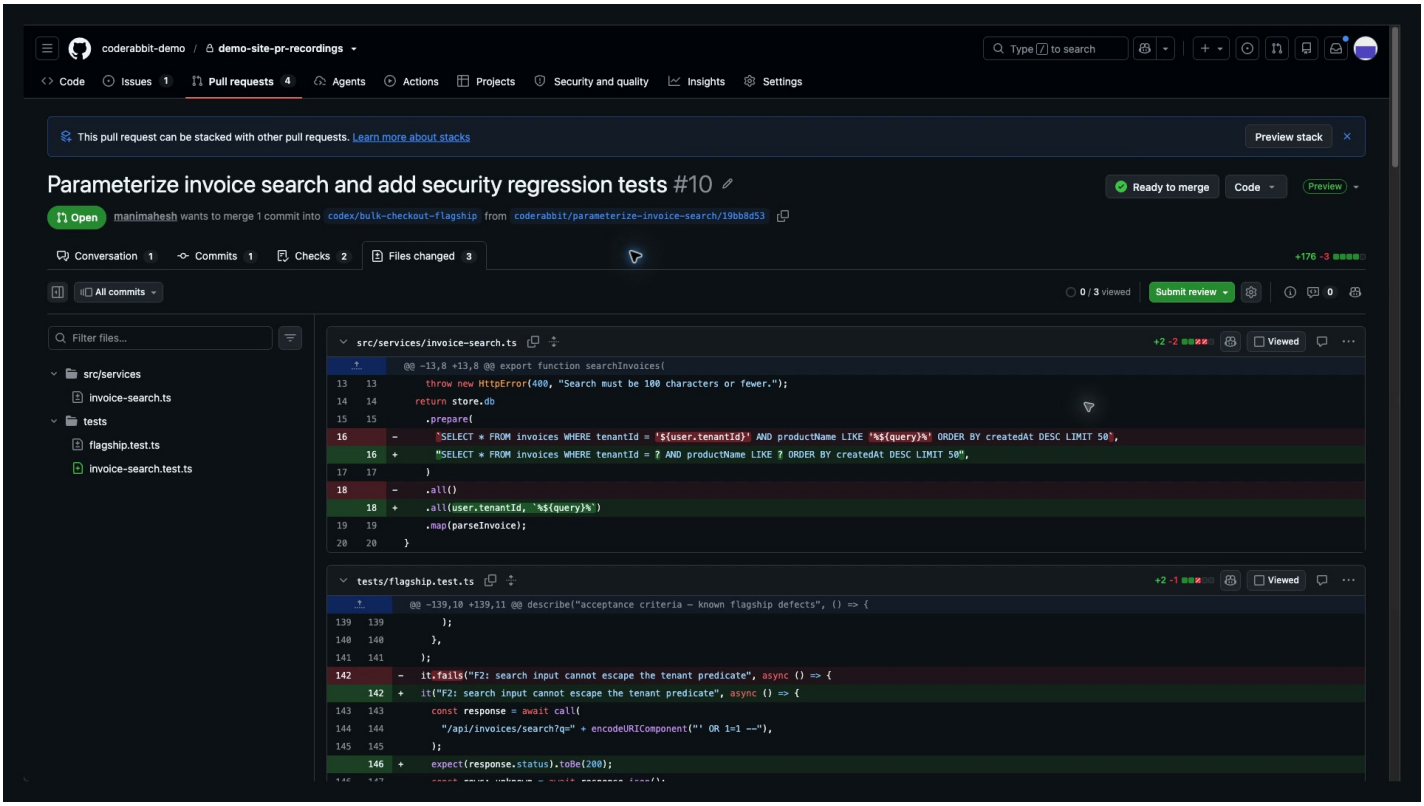
Compare the delivered patch with the task's Changes view.

PR #10: files changed

SAY

I check that the pull request contains the expected query change and tests. These are the changes we inspected in the task, now available to the team.

SEE / VIDEO FRAME AT 04:12



DELIVER / STEP 4 OF 4 / 04:15-04:34

Read validation and review status

DO

Return to Conversation and read the delivery summary and skipped-review explanation.

Use the task link for earlier test evidence; the delivery did not rerun validation.

Leave the PR open and return to the flow slide.

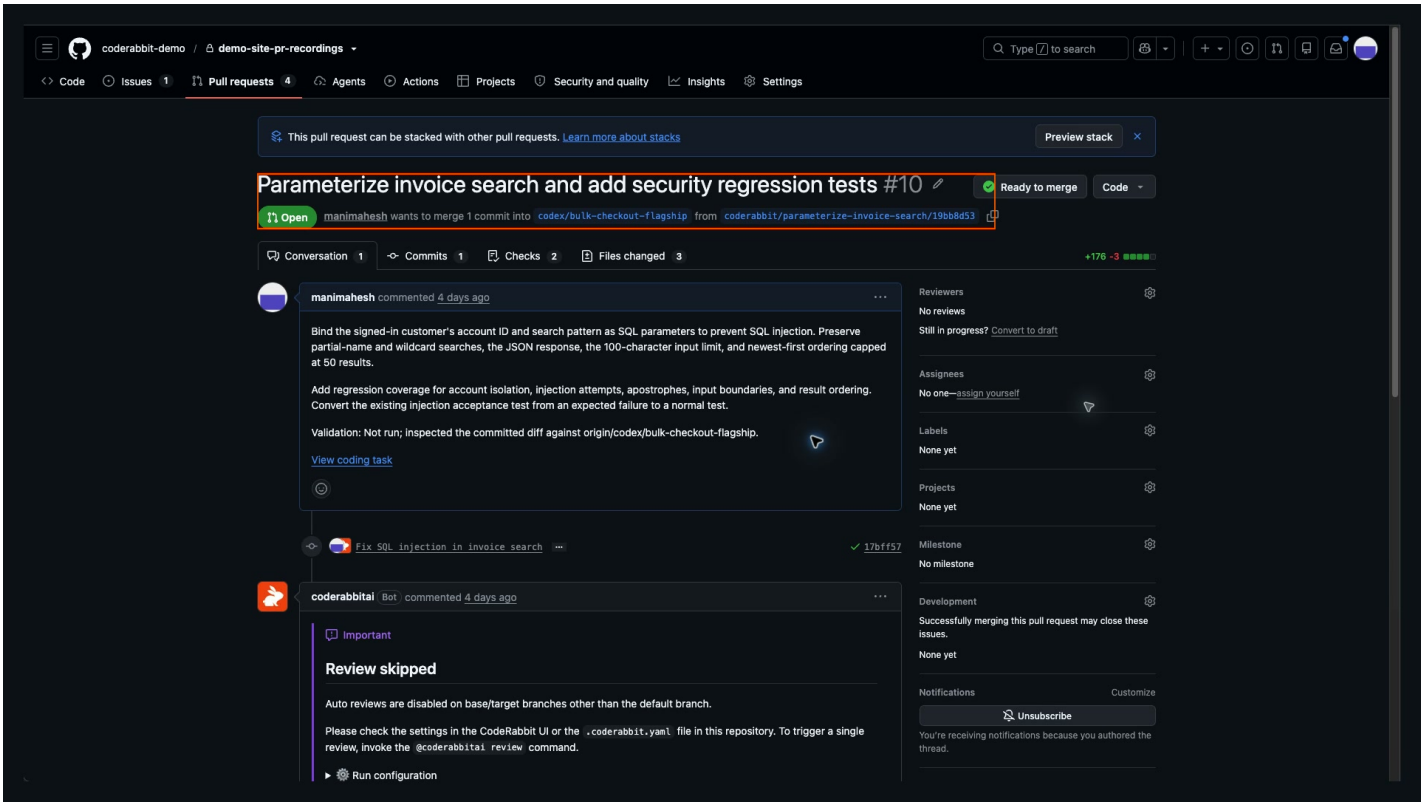
Recorded Coding Agent task

Coding Agent result: PR #10

SAY

The delivery step didn't rerun validation. The earlier results are recorded in the coding task. Automatic review was also skipped for this feature branch destination. We've scoped the work, reviewed a plan, followed the implementation and inspected the patch before delivery. The pull request remains open for the team's review.

SEE / VIDEO FRAME AT 04:31



Operator text to type

Use these inputs for the Do instructions. They are separate from the spoken Say track.

Recorded coding-task request

Fix SQL injection in invoice search. Bind the signed-in customer's account ID and search text as SQL parameters. Preserve partial-name searches, the API response, the 100-character input limit, newest-first order, and the 50-result limit. Add tests proving normal searches work and crafted input cannot expose another customer's invoices. Convert the existing expected-failure test for this issue into a normal passing test. Keep discounts and invoice exports unchanged. First propose a plan and wait for approval. Do not commit, push, or open a PR; I will deliver the reviewed patch manually.

Mid-task test request

Before finishing, verify that a product name containing an apostrophe remains searchable, and that a matching product in another customer account is excluded. If these tests are already present, confirm their results.

Resource directory

Links open the recorded examples. Historical comments and pinned source revisions preserve the relevant context when the live PR has moved on.

PR #4: SQL finding
https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4#discussion_r3991696811

Recording repository
https://github.com/coderabbit-demo/demo-site-pr-recordings

Recorded Coding Agent task
https://app.coderabbit.ai/code/tasks/19bb8d53-3b68-4a6a-ba65-29125b8e134b

Coding Agent result: PR #10
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/10

Slide deck
https://docs.google.com/presentation/d/1zk6UD0ZoAxtGRqDTQL-MVulAWu5MH4nYmd-9xJBpU5Y/edit

PR #10: files changed
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/10/files